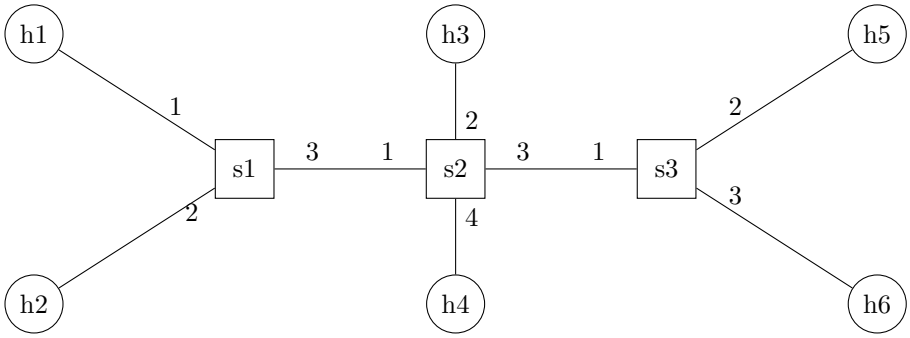
Piattaforme Software per la Rete (Mod. 1) – 27/07/2016

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura.



(a) Scrivere il contenuto delle tabelle openflow dopo l'accensione di tutti i nodi.

Soluzione:

s1

Priority	Match	Action	pkt cnt
0	*	output controller	0
1	eth_dst = MAC1 eth_type = 0x800	output 1	0

s2

Priority	Match	Action	pkt cnt
0	*	output controller	0
1	eth_dst = MAC1 eth_type = 0x800	output 1	0
2	eth_dst = MAC3 eth_type = 0x800 ip_dst = IP3	set eth_dst=MAC4 set ip_dst=IP4 ouput 4	0
2	eth_src = MAC4 eth_type = 0x800 ip_src = IP4	set eth_src=MAC3 set ip_src=IP3 ouput flood	0

s3

Priority	Match	Action	pkt cnt
0	*	output controller	0
1	eth_dst = MAC1 eth_type = 0x800	output 1	0

Programma 1 (1/2) – Switch Features

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    match = parser.OFPMatch(
        eth_dst=MAC1,
        eth_type=0x0800)    # IPv4
    actions = [ parser.OFPActionOutput(1) ]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    if (datapath.id == 2):
        match = parser.OFPMatch(
            eth_dst=MAC3,
            eth_type=0x0800,    # IPv4
            ip_dst=IP3)
        actions = [
            parser.OFPActionSetField(eth_dst=MAC4),
            parser.OFPActionSetField(ip_dst=IP4),
            parser.OFPActionOutput(4)
            ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=2,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

        match = parser.OFPMatch(
            eth_src=MAC4,
            eth_type=0x0800,    # IPv4
            ip_src=IP4)
        actions = [
            parser.OFPActionSetField(eth_src=MAC3),
            parser.OFPActionSetField(ip_src=IP3),
            parser.OFPActionOutput(ofproto.OFPP_FLOOD)
            ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=2,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

Programma 1 (2/2) – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp)
    pkt_tcp = pkt.get_protocol(tcp.tcp)

    if (pkt_tcp is not None) and (pkt_tcp.dst==80):
        match = parser.OFPMatch(
            eth_dst=pkt_eth.src,
            eth_type=0x0800,    # IPv4
            ip_proto=6)         # TCP
        actions = [ parser.OFPActionOutput(in_port) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    data = msg.data
    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

L'host h5, porta 3456 invia un datagramma UDP a h1, porta 123.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1 e s2.

Soluzione:
$s2 \rightarrow s1$: MAC5 > MAC1 IP5 > IP1 UDP 3456 > 123
$s1 \rightarrow h1$: MAC5,MAC1,IP5,IP1,UDP,3456,123

L'host h1, porta 5432 invia una richiesta di apertura di connessione a h3, porta 123.

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow h4$: MAC1,MAC4,IP1,IP4,TCP,5432,123 [SYN]

Il server risponde.

(d) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow s1$,$s3$,$h3$: MAC3,MAC1,IP3,IP1,TCP,5432,123 [SYN ACK]

L'host h2, porta 5432 invia una richiesta di apertura di connessione a h6, porta 80. Nel caso il pacchetto non arrivi a destinazione ripetere l'invio altre 2 volte.

(e) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow \mathcal{C}$: packet_in (MAC2,MAC6,IP2,IP6,TCP,5432,80 [SYN])
$s2 \rightarrow s1$,$s2$,$h3$,$h4$: MAC2,MAC6,IP2,IP6,TCP,5432,80 [SYN]

Il server risponde.

(f) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow s1$: MAC6,MAC2,IP6,IP2,TCP,80,5432 [SYN ACK]

(g) Scrivere il contenuto finale delle tabelle openflow.

Soluzione:

s1

Priority	Match	Action	pkt cnt
0	*	output controller	?
1	eth_dst = MAC1 eth_type = 0x800	output 1	?
3	eth_src = MAC6 eth_type = 0x800 ip_proto = 6	output 2	?

s2

Priority	Match	Action	pkt cnt
0	*	output controller	?
1	eth_dst = MAC1 eth_type = 0x800	output 1	?
2	eth_dst = MAC3 eth_type = 0x800 ip_dst = IP3	set eth_dst=MAC4 set ip_dst=IP4 ouput 4	?
2	eth_src = MAC4 eth_type = 0x800 ip_src = IP4	set eth_src=MAC3 set ip_src=IP3 ouput flood	?
3	eth_src = MAC6 eth_type = 0x800 ip_proto = 6	output 1	?

s3

Priority	Match	Action	pkt cnt
0	*	output controller	?
1	eth_dst = MAC1 eth_type = 0x800	output 1	?
3	eth_src = MAC6 eth_type = 0x800 ip_proto = 6	output 1	?

Esercizio 2

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
64.0.0.0/2	*	80	*	TCP
64.0.0.0/3	128.0.0.0/2	80	22	TCP
*	96.0.0.0/3	*	53	UDP
64.0.0.0/2	*	*	123	UDP
128.0.0.0/2	64.0.0.0/2	23	*	UDP
96.0.0.0/3	64.0.0.0/3	25	80	*

(a) Definire il bitvector set per ciascun campo.

Soluzione:

Source Address	
64.0.0.0/2	101100
64.0.0.0/3	111101
96.0.0.0/3	101101
128.0.0.0/2	001010
*	001000

Destination Address	
64.0.0.0/2	100110
64.0.0.0/3	100101
96.0.0.0/3	101010
128.0.0.0/2	110100
*	100100

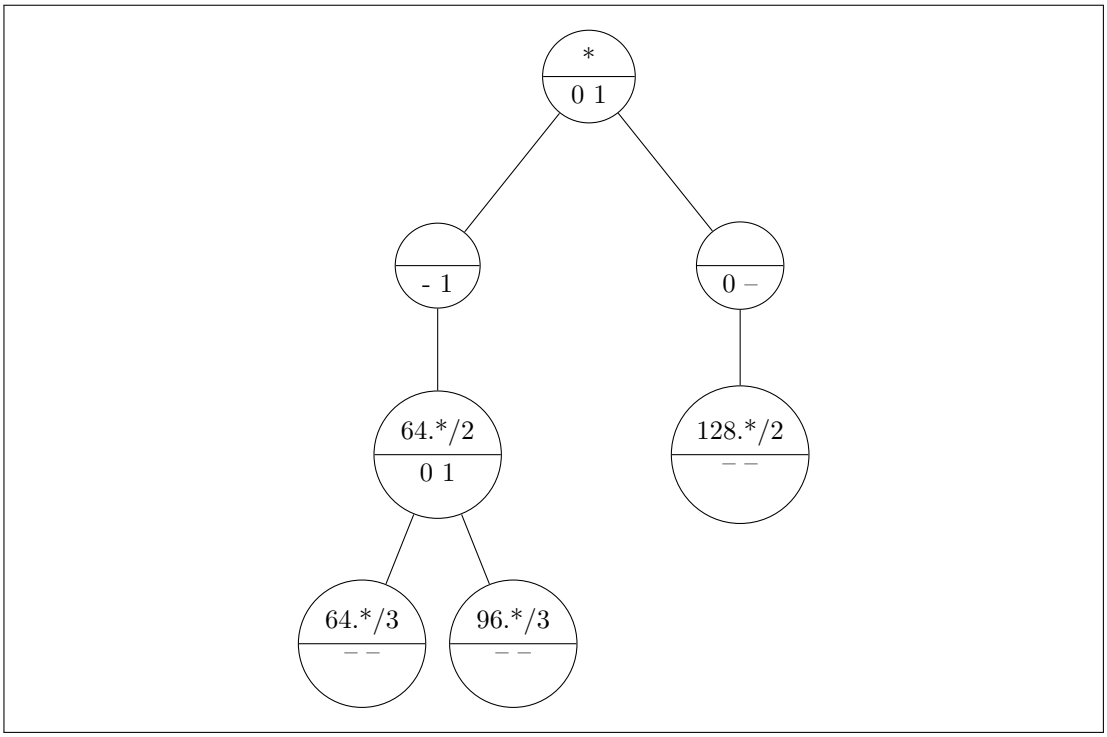
Source Port	
23	001110
25	001101
80	111100
*	001100

Dest. Port	
22	110010
53	101010
123	100110
80	100011
*	100010

protocol	
TCP	110001
UDP	001111
*	000001

(b) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando uno unibit trie. Disegnare il trie.

Soluzione:



(c) Si considerino i pacchetti (64.10.0.1, 96.0.0.2, 80, 53, UDP) e (97.50.0.1, 96.0.0.2, 80, 123, TCP). Applicare ad entrambi la procedura di BVLS per identificare gli eventuali match.

Soluzione: 1) 111101 AND 101010 AND 111100 AND 101010 AND 001111 = 001000
La terza regola.
2) 101101 AND 100101 AND 111100 AND 100110 AND 110001 = 100000
Il pacchetto soddisfa la regola 1.

(d) Si dica qual è la profondità massima dello unibit trie nei due casi IPv4 e IPv6.

Soluzione: Nel caso IPv4 32 bit. Nel caso IP64 128 bit.

Esercizio 3

Una matrice di commutazione crossbar di uno switch 4×4 usa un algoritmo per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza L,

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

Ingresso 1: 4, 2, 1

Ingresso 2: 2, 2, 3

Ingresso 3: –, 4, 3

Ingresso 4: –, 4, 1

(a) Svolgere l'algoritmo Take-a-Ticket (TaT) fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali ticket sono assegnati. L'algoritmo di assegnazione dei ticket privilegia sempre l'interfaccia di ingresso con l'identificativo più piccolo.

Soluzione:

Round		Ingresso 1	2	3	4
1	Assegnazione	(1,1)	(3,1)	(3,2)	(1,2)
	Trasmissione	1	3	–	–
2	Assegnazione	(2,1)	(2,2)	–	–
	Trasmissione	2	–	3	1
3	Assegnazione	(4,1)	–	(4,2)	(4,3)
	Trasmissione	4	2	–	–
4	Assegnazione	–	(2,3)	–	–
	Trasmissione	–	2	4	–
5	Assegnazione	–	–	–	–
	Trasmissione	–	–	–	4

(b) Facendo riferimento all'algoritmo TaT con assegnazione casuale, si consideri uno scenario di traffico uniforme e code sempre sature. Qual è la probabilità che tutte le uscite siano usate?

Soluzione:

$$p = \frac{4!}{4^4} = \frac{3}{32} \simeq 9\%$$

(c) Si consideri il caso in cui i pacchetti diretti alla coda 3 abbiano lunghezza $2L$. Ripetere la simulazione dell'algoritmo TaT.

Soluzione:

Round		Ingresso 1	2	3	4
1	Assegnazione	(1,1)	(3,1)	(3,2)	(1,2)
	Trasmissione	1	3	–	–
2	Assegnazione	(2,1)	–	–	–
	Trasmissione	2	(3)	–	1
3	Assegnazione	(4,1)	(2,2)	–	(4,2)
	Trasmissione	4	2	3	–
4	Assegnazione	–	(2,3)	–	–
	Trasmissione	–	2	(3)	4
5	Assegnazione	–	–	(4,3)	–
	Trasmissione	–	–	4	–